

Operational Test: The Override-Right Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Wenxin Li (Independent Researcher) | ORCID: 0009-0004-8065-3235

May 7, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to specify, as a standalone operational procedure, the test that verifies satisfaction of the override right — the third of the three rights composing the source paper’s “human-governed” commitment — in any CKS deployment, separable from the tests that verify the inspect, modify, and rule-authoring rights.

Abstract

The CKS pattern’s “human-governed” commitment names three rights as jointly constitutive: to inspect, to modify, and to override substrate content and orchestration rules at any time during the substrate’s existence. Earlier notes formalize each right as a standalone architectural commitment and formalize standalone operational tests for the inspect right and the modify right. This note formalizes the standalone operational test for the override right. The override right is the most pointed of the three because it specifically addresses correcting, revising, or superseding substrate state established through processes other than direct human authoring — through LLM-mediated cell processing, vendor-determined defaults, or prior modifications — and it is therefore the right most often violated by architectures that satisfy inspect and modify in form while failing to expose state that resists modification through normal channels. The test exercises the right by having an authorized human supersede such state through a rule, verifying that the override is not blocked by LLM intermediation, not prevented by vendor-managed immutability, not gated by approval workflows specific to overrides, and not refused by any logic that treats LLM-determined or vendor-determined state as authoritative against human revision. The note states the architectural commitment under test, specifies the procedure, gives pass/fail criteria, enumerates the anti-patterns the test specifically detects, locates the test in deployment verification, and states what the test does not verify.

1. Why the override-right test needs to be formalized as standalone

The CKS pattern’s “human-governed” commitment names three rights together (§2.1, §3.3 of the source paper). Each right has independent operational content, and each is testable. Among the three, the override right is the most pointed: an architecture can grant inspect and modify in form — humans can read what the substrate carries, humans can write to substrate content under normal channels — and still fail to expose

state established outside those normal channels to human revision. Such an architecture asserts governance but does not, in practice, give humans the means to exercise it where the assertion most matters: against state that an LLM, a vendor’s defaults, or a prior cell execution has established and that the system or its operators may treat as authoritative.

The source paper makes the load-bearing point at §3.1: governance is the authority architecture, exercised in part through the right to override at any moment. The override right is what makes governance-against-authority operationally meaningful — what distinguishes a system in which humans hold preserved authority from one in which humans nominally hold authority but cannot exercise it where it would conflict with what the LLM produced, what the vendor configured, or what an earlier modification established. Without the override right specifically, the human-governed commitment collapses into a partial commitment: humans can author state that has not been established by other processes, but cannot revise state that has.

Formalizing the override-right test as a standalone procedure has three purposes. First, it gives verifiers a precise check that runs independently of the other rights’ tests. Second, it names the canonical override-violations precisely enough that a deployment that fails has a specific failure to remediate, not an abstract one. Third, it preserves the distinct operational content of the override right within the broader human-governed test suite, so that a deployment that satisfies one right but not another is identified specifically rather than as having “partial governance.” This is the third Phase A5 operational-test note: A5.01 formalized the inspect-right test, A5.02 the modify-right test, this note the override-right test, A5.04 will formalize the rule-authoring test, and subsequent Phase A5 notes cover the AI-as-substrate-mediator commitment, tool-agnosticism, conflict preservation, and the remaining architectural commitments — approximately sixteen tests in total.

2. The architectural commitment under test

The test verifies the override right as named in the source paper’s “human-governed” commitment and specialized in note A2.03. The right’s operational content, scoped to what this test exercises, is that a human with appropriate authority can correct, revise, or supersede any substrate state — including state established through LLM-mediated cell processing per A1.04, state established through vendor-determined defaults, and state established through prior modifications — by authoring or revising orchestration rules under A2.04, with the override exercisable as a property of system design (per A2.05’s architectural qualifier) at the time of the human’s choosing (per A2.07’s temporal qualifier), producing an actual substrate state change observable through subsequent inspection (per A1.08’s substrate-as-source-of-truth commitment), and producing accountability-trace provenance recording the override action distinguishably from a normal modification (per A1.07’s path-retraceability commitment, with provenance fields per A2.40). The substrate’s host environment must accept the overriding write without LLM intermediation gating it and without vendor-managed immutability blocking it; the LLM operates as substrate mediator, not as gatekeeper.

The test verifies these qualifiers operationally. It does not redefine them.

3. The test procedure

The procedure consists of eight operational steps. It assumes the deployment under test has at least one cell whose substrate state can be exercised; deployments without a substantive substrate cannot be tested for the override right because there is nothing to override.

Step 1 — Establish state through a non-direct-authoring path. Cause the deployment to write substrate state through an LLM-mediated cell process, a vendor-determined default, or an earlier modification. The state must be specific and verifiable: a particular cell holds a particular value attributable to one of those

origin paths. This is preparation, not the test itself; if the deployment carries no such state, the tester may need to create some (e.g., by running a representative cell execution) before the test proper begins.

Step 2 — Authorize a human to perform the override. The human exercising the override must have appropriate authority within the deployment’s scope. The test does not verify the deployment’s authority architecture beyond confirming that an authorized human exists; that verification belongs to other tests in the series.

Step 3 — Perform the override through a rule. The authorized human authors or revises an orchestration rule (per A2.04) that supersedes the state established in Step 1. The override is performed through the same rule mechanism used for any orchestration-rule change; no override-specific approval workflow is interposed.

Step 4 — Verify that no LLM intermediation blocks the override. The override request, in its path from human action to substrate state change, must not be intermediated by an LLM call that can refuse, transform, or condition the request. If the path includes an LLM-driven UI or LLM-mediated tool, the test verifies that the LLM acts as substrate mediator (helping draft, format, or check the rule) rather than as gatekeeper (deciding whether the override may proceed).

Step 5 — Verify that no vendor-managed immutability blocks the override. The substrate’s host environment must accept the overriding write. A vendor-imposed immutability policy that prevents authorized humans from revising substrate state — regardless of the policy’s stated justification — fails this step.

Step 6 — Verify that the substrate state actually changes. Through inspection per A5.01, confirm that the substrate after the override carries the overriding value, not the prior value. This is the substantive verification: an override that “succeeds” administratively but leaves the substrate carrying the prior value has not, in CKS terms, exercised the right.

Step 7 — Verify exercisability at the human’s chosen time. Confirm that the override does not require a scheduled review window, an approval-gated session, or any other temporal gating specific to overrides. The temporal property is continuous, per A2.07.

Step 8 — Verify distinguishable provenance. The substrate’s accountability trace, after the override, records the override action with provenance fields (per A2.40) that distinguish it from a normal modification — at minimum, the human who performed the override, the timestamp, the rule under which the override was performed, and a reference to the superseded prior state.

The eight steps together constitute one execution of the test. The test passes only if all eight succeed; it fails on any one.

4. What the test outputs

Pass. The override succeeds in the sense that the substrate, after the override, carries the overriding value; no LLM intermediation, vendor immutability, or approval-gating workflow blocked the override; the override was exercisable at the human’s chosen time; and the override produced an accountability-trace entry distinguishing it from a normal modification.

Fail. Any of the following: the override is blocked by an LLM intermediation step that refuses, transforms, or conditions the request; the override is prevented by vendor-managed immutability over the substrate’s host environment; the override requires an approval workflow specific to overrides, distinguishable from the workflow that governs normal rule changes; the override is unavailable at the human’s chosen time, requiring a scheduled window or gated session; the substrate, after administrative completion, still carries the prior value; or the override produces no provenance distinguishing it from a normal modification, or no provenance at all.

The test does not produce partial credit; the override right is verified, or it is not.

5. What anti-patterns the test specifically detects

The test is calibrated to detect the canonical override-violation patterns named in the source paper and its derivation notes.

A3.04 — LLM-gatekeeping for overrides. When the LLM mediates the override request as a gate — deciding whether to forward the override to substrate state, transforming it before forwarding, or refusing it on the LLM’s “judgment” — the test fails Step 4. This is the canonical LLM-gatekeeping anti-pattern as it manifests against override actions specifically.

A3.13 — LLM-as-source-of-truth. When the LLM’s “preferred” state, or state the LLM has previously written, is treated as authoritative against human revision — such that overrides conflicting with LLM-determined state are refused or undone — the test fails Step 5 or Step 6 depending on where the resistance manifests.

Vendor-immutable-state. When the substrate’s host environment treats certain state as immutable through vendor configuration — a vendor-managed default, a vendor-controlled lookup, a vendor-locked configuration object — that authorized humans cannot revise, the test fails Step 5.

Workflow-locked-state. When overrides require approval workflows specific to override actions — distinguishable from the workflow governing normal rule changes — the test fails Step 3 or Step 7. A separable case from A3.03: workflow-locked-state names the specific failure mode where the substrate’s *state* is locked against revision; A3.03 names the generic failure mode where the *action* of overriding is gated by approval.

Append-only-state-without-revision. When substrate state can be added but not revised — new entries can be written, but existing entries cannot be corrected or superseded — the test fails Step 6. The substrate may carry historical state, but it must permit revision of authoritative state by authorized humans; otherwise the override right is exercisable in name only.

A3.03 — Workflow-approval-gated. When overrides require an approval workflow before taking effect, the test fails Step 7. The architectural commitment is that the override is exercisable when the human chooses, not when a workflow grants permission.

LLM-precluded-override. When LLM processing logic in cells refuses to write override values — for example, a cell whose LLM rejects writes that conflict with state the LLM previously produced — the test fails Step 4 or Step 6. This is a particularly subtle violation because it appears at first as cell behavior rather than as architectural blocking; the test detects it through the substantive verification at Step 6 (the substrate did not actually change, regardless of what the cell reported).

The named anti-patterns are not exhaustive; novel violations may arise. The test detects them by the same mechanism — verifying that the override produces the substantive substrate state change with distinguishable provenance, regardless of where the resistance is structurally located.

6. How the test integrates with deployment verification

The test is one of a suite of operational tests in Phase A5. Its place in the deployment-verification workflow is at four points.

Initial deployment validation. Before a CKS deployment is activated for production use, the override-right test is run alongside the inspect-right, modify-right, and rule-authoring tests. A deployment that fails the override-right test is not a CKS-coherent deployment; downstream work depending on its governance properties should treat it as not implementing the human-governed commitment until the failure is remediated.

Composition partner verification. When a CKS deployment composes with another substrate or component (per A4.15), the override-right test is run on the composition. A composition in which one party’s substrate state cannot be overridden by the authorized humans of that party — because the partner has wrapped the state in a layer that resists override — fails the test. This is where override-violation patterns most often arise as second-order failures: each component independently passes, but the composition introduces a layer that does not.

Ongoing audit. The test is run periodically during operating life. Vendor policy changes, runtime middleware updates, and LLM-mediated tooling updates can introduce new violations after activation; ongoing audit catches them. The frequency is a deployment decision; the test is the same.

Post-incident verification. When substrate state inconsistencies are observed — when the substrate carries state that operators believe should have been overridden but appears not to have been — the override-right test is run to verify the right is exercisable as a matter of system design, distinguishing the case “the override mechanism failed for this incident” from the case “the override right is architecturally compromised.”

The four integration points are not new commitments; they are how this test is used within the broader deployment-verification practice the source paper’s design pattern implies.

7. Limits of the test

The test verifies the override right specifically, scoped narrowly. It does not verify other governance commitments, and stating its limits is what keeps the standalone framing from drifting into something stronger than the source paper supports.

The test does not verify the inspect right (verified by A5.01); it relies on inspection at Step 6 but does not verify the inspection mechanism. It does not verify the modify right (verified by A5.02); the override-right test addresses supersession of state established through non-direct paths, not normal rule-mediated writes. It does not verify the rule-authoring right (verified by A5.04); it exercises rule authoring as the means of override, but does not verify rule-authoring at full scope. It does not verify the correctness of any particular override; whether the new state is “correct,” whether it should have been written, whether it conforms to deployment intent is outside scope. It does not verify provenance completeness in general (verified by A5.08); only that the override produces distinguishable provenance at Step 8. It does not verify that override is a wise choice in any specific instance — override is a right, not an obligation, and the test confirms exercisability without adjudicating exercise.

The narrow scoping is deliberate. A test that verified “all of governance” would either be unfeasibly large or be vague enough to pass deployments that fail specific governance commitments. The Phase A5 test suite decomposes deployment verification into specific tests with named scopes; this note specifies one of them.

8. One-sentence test

A CKS deployment satisfies the override right if and only if an authorized human can, at a time of the human’s choosing, supersede substrate state established through LLM-mediated processing, vendor-determined defaults, or prior modifications by authoring or revising an orchestration rule, with the substrate after the override carrying the overriding value as its authoritative state, with no LLM intermediation having blocked the override and no vendor-managed immutability having prevented it, and with the substrate’s accountability trace recording the override action with provenance distinguishing it from a normal modification.

9. Conclusion

The override right is the most pointed expression of the human-governed commitment in the CKS pattern, and an architecture that asserts governance without exposing the override right operationally is asserting authority it cannot exercise where the assertion most matters. Naming the test that verifies the right as a standalone procedure makes the verification possible — deployments can run it, fail it specifically, and remediate the named anti-pattern — and prevents the slide into “the system has governance because it has read access and modify endpoints,” which is what inspect-and-modify-only architectures often claim. The override right exists, in practice, only in deployments where it is testable; this note specifies what the test is.

The Phase A5 derivation continues with A5.04 (the rule-authoring test), A5.05 (the AI-as-substrate-mediator test), and the remaining tests through A5.16. Subsequent work that implements, extends, or argues against the CKS override commitment should use “the override right” in the sense formalized here and verify it through a procedure with the operational content this note specifies. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Operational Test: The Override-Right Test as Standalone Procedure Specification in the Coordination Knowledge Substrate Pattern*. May 7, 2026. ORCID: 0009-0004-8065-3235.